

Formatos de mensajes de texto

Redes de Comunicaciones - Curso 2023/24

Introducción

Durante esta sesión vamos a analizar distintas formas posibles para especificar un protocolo de comunicación. No hay una forma óptima de llevar a cabo la especificación de un protocolo puesto que esta dependerá del propósito del mismo, de los dispositivos involucrados, de las propiedades de la conexión de red, etc. Para un mismo protocolo puede haber distintas formas correctas de llevar a cabo la especificación.

El diseño del protocolo puede dividirse en dos partes bien diferenciadas, que no necesariamente deben realizarse por este orden:

- Definir el formato de los mensajes intercambiados entre las entidades participantes.
- Definir la temporización o secuencia correcta de intercambio de dichos mensajes, que normalmente suele especificarse mediante modelos computacionales más complejos como son, por ejemplo, los autómatas de estados finitos.

En este boletín nos centraremos en el formato de los mensajes, concretamente en el uso de mensajes basados en texto. Dicho formato textual será el que los alumnos deben utilizar a la hora de definir los mensajes de su protocolo de comunicación con el directorio del sistema *NanoFiles*.

Mensajes en formato *field:value*

Aunque existen varias posibilidades a la hora de especificar el formato de los mensajes de texto, **en este boletín nos centraremos en el formato campo:valor (*field:value*)**. El formato *field:value* utiliza delimitadores basados en texto y facilita la legibilidad de los mensajes, es decir, son fáciles de interpretar a simple vista. Este formato es usado en protocolos tan importantes como HTTP, FTP o SMTP. Un ejemplo de uso de dicho formato es el siguiente mensaje, que se podría usar para solicitar ~~al servidor la descarga de un fichero, identificado por su hash:~~ **login por parte del cliente**

```
operation: login\nnickname: alicia\n\n Fin del Mensaje
```

- Cada línea es un campo del mensaje, con un nombre determinado (generalmente representativo de la información que representa) y su valor asociado.
- El primer campo suele indicar el significado del mensaje. Así, en el ejemplo anterior, el significado del campo "operation" indica que se trata de un mensaje de *login*, como el que se podría enviar al directorio para iniciar sesión con un cierto nombre de usuario.
- Al final de cada línea debe enviarse un retorno de carro ('\n') para marcar el final de la línea, mientras que una línea vacía (con sólo un retorno de carro) marca el final del mensaje.
- Puesto que existe una forma explícita de indicar el fin del mensaje, el número de campos (líneas) puede ser variable, por lo que un determinado campo puede aparecer varias veces, o no hacerlo en absoluto.

- En este tipo de mensajes debe evitarse el uso de los delimitadores como parte del valor del propio texto del campo (en nuestro ejemplo, el retorno de carro '\n'). Existen varias técnicas para resolver este problema, por ejemplo, excluir el retorno de carro de los caracteres posibles de uso dentro del texto o codificar el retorno de carro por otro carácter no usado en el texto de los mensajes (por ejemplo, &, # ó %).

Un inconveniente de este tipo de formato es que no permite anidar elementos, si bien existen formas alternativas de representar datos compuestos. Por ejemplo, el siguiente mensaje podría usarlo el directorio para enviar a un cliente de NanoFiles la lista de usuarios que se han registrado en el directorio:

```
operation: registeredUsers\n
users: alicia,pedro,juan\n
\n
```

En caso de usar un carácter como delimitador de elementos que conforman el valor de un campo compuesto, se ha asegurar que dicho delimitador no aparece como valor de ningún elemento. En el ejemplo anterior en el que el valor del campo es una lista de nombres de fichero separados por comas, los *nicknames* de los usuarios no podrán incluir comas y, en caso de que sea necesario, dicho carácter ',' se habrá de codificar de forma especial para no interpretarlo como un delimitador cuando no corresponda.

A la hora de representar datos compuestos, también puede resultar útil que un mismo campo aparezca en repetidas ocasiones en el mismo mensaje. En este caso, el orden de los campos puede usarse para establecer la correspondencia entre los diferentes valores que conforman un mismo dato compuesto:

```
operation: registeredUsers\n
user: alicia\n
isServer: true\n
user: pedro\n
isServer: false\n
user: juan\n
isServer: false\n
\n
```

[userlistok](#) , y indica quien esta sirviendo ficheros

Implementación de los mensajes en el proyecto NanoFiles

En este apartado se proporcionan indicaciones generales acerca de cómo crear y procesar los mensajes transmitidos entre el cliente y el servidor de directorio a través de datagramas.

En el marco del proyecto de NanoFiles, el paquete `nanoFiles.udp.message` contiene la clase `DirMessage.java` , que esboza las líneas generales acerca de cómo se podrían implementar mensajes textuales definidos en el protocolo de comunicación con el directorio.

Creación de mensajes `DirMessage` para su posterior envío

La clase `DirMessage` contendrá como atributos los valores de los campos de los mensajes del protocolo, tales como el tipo de operación, nombres de usuario, metadatos de ficheros (nombre, tamaño, hash),

números de puerto, direcciones de socket de un servidor, etc.

Por tanto, será necesario añadir un atributo a la clase por cada uno de los campos de los diferentes tipos de mensajes definidos para este protocolo. En el código de partida ya existe, entre otros, un campo `operation` que se debe usar para indicar en el constructor el tipo de mensaje a crear. Tras crear un mensaje de un tipo de operación dado, el código "cliente" de la clase `DirMessage` (es decir, las clases `DirectoryConnector` y `NFDirectoryServer`) se encargará de establecer mediante métodos *setter* los atributos necesarios con la información que se desea enviar en el mensaje.

NOTA: Es recomendable que los métodos *setter* para establecer los atributos del objeto `DirMessage` comprueben que dicho atributo es relevante para el tipo de mensaje en cuestión, con el fin de detectar cuándo se está tratando de establecer un atributo que no corresponde a ningún campo definido para el tipo de mensaje.

Conversión de `DirMessage` a `String` para envío de mensajes textuales

Los mensajes de este protocolo siempre van a ser transmitidos como cadenas de caracteres. Para ello tendremos que implementar en la clase `DirMessage` el método `toString()` que se encargue de generar realmente el mensaje en formato `String` a partir de los valores de los atributos del objeto de la clase `DirMessage`. La forma de hacerlo dependerá del formato de mensaje que hayamos elegido, tal como se ha visto en la sección anterior.

Los protocolos basados en *field:value* se caracterizan porque sus mensajes están compuestos por líneas en las que aparece en cada línea el identificador del campo y el valor del mismo, separados por el carácter dos puntos. Para construir una cadena con el mensajes en este formato tendremos que ir generando, línea por línea, los campos del mensaje que queremos transmitir con sus correspondientes valores.

Para la construcción línea por línea podemos hacer uso de un `StringBuffer`. Supongamos que queremos enviar un mensaje que tiene dos campos. El código de abajo nos da un ejemplo de cómo hacerlo:

```
final char DELIMITER = ':';    //Define el delimitador
final char END_LINE = '\n';    //Define el carácter de fin de línea
StringBuffer sb = new StringBuffer();
String field1 = "operation";    //Nombre del campo
String value1 = "login";       //Valor del campo
String field2 = "nickname";    //Nombre del campo
String value2 = "alicia";      //Valor del campo

sb.append(field1+DELIMITER+value1+END_LINE); //Construimos el campo
sb.append(field2+DELIMITER+value2+END_LINE); //Construimos el campo
sb.append(END_LINE);           //Marcamos el final del mensaje
String message = sb.toString(); //Se obtiene el mensaje
```

Creación de objetos `DirMessage` a partir de un `String` recibido

La clase `DirMessage` cuenta con método de clase llamado `fromString(String str)` que será utilizado por el extremo receptor de un datagrama. Este método se deberá encargar, partiendo de una cadena de caracteres recibida en un datagrama, de interpretar su contenido y construir un objeto de clase

`DirMessage` con los valores extraídos del mensaje. La cadena habrá sido generada mediante el método `toString` antes de ser enviada en el datagrama por el emisor. Así, el método `fromString` nos permite crear un objeto de dicha clase `DirMessage`, en el cual los valores de los distintos campos del mensaje contenido en el `String` hayan sido guardados en los correspondientes atributos del objeto.

Existen diversas alternativas para procesar los mensajes de un protocolo basado en *field:value*. Una de las más elementales es usar el método `split()` de la clase `String` con el delimitador adecuado (`"\n"`) para obtener un array con todas la líneas, sobre el que ir iterando posteriormente.

A continuación dejamos un extracto de código que nos permitiría procesar

```
String[] lines = message.split(END_LINE + "");
for (String line : lines) {
    int idx = line.indexOf(DELIMITER); // Posición del delimitador
    String fieldName = line.substring(0, idx).toLowerCase(); //
minúsculas
    String value = line.substring(idx + 1).trim();
    // DO SOMETHING WITH fieldName AND value
    ...
}
```

Ejercicios propuestos

Para avanzar en el desarrollo de la práctica, has de tener en cuenta la funcionalidad prevista para el directorio, según los requisitos de la práctica. Para más detalles, consulta el documento de enunciado de *NanoFiles* con la especificación, disponible en el Aula Virtual.

La funcionalidad mínima obligatoria que debe soportar el protocolo con el directorio se limita a iniciar sesión en el directorio con un *nickname* único, obtener la lista de usuarios registrados y cerrar sesión. En todos los mensajes enviados al directorio tras iniciar sesión se debe proporcionar la clave de sesión obtenida.

Entre las mejoras propuestas, aquellas que implican dotar de más funcionalidad a este protocolo están las siguientes:

- Dar de alta en el directorio un servidor de ficheros en un puerto, asociado a su *nickname*.
- Consultar la dirección en la que escucha un servidor a partir de su *nickname*
- Publicar al directorio la lista de ficheros compartidos.
- Consultar la lista de los ficheros publicados al directorio por los servidores.
- Averiguar los servidores que comparten un determinado fichero.
- ...

Teniendo en cuenta lo anterior, a continuación se proponen una serie de ejercicios para avanzar en el diseño e implementación de la práctica.

1. Haciendo uso de un documento de texto plano, **diseña** el formato del mensaje necesario para iniciar sesión con un nombre de usuario en el directorio.

1. Debes definir los campos (líneas en formato *field:value*) que contiene cada mensaje, empezando por la operación que lleva a cabo. Indica entre otros aspectos quién envía y quién recibe dicho mensaje, su significado, en qué situaciones se envía, y qué acciones se llevan a cabo cuando se recibe. Pon ejemplos de cada mensaje, usando valores ficticios para ello como se ha visto anteriormente en el boletín.
2. Define el formato del mensaje de respuesta que contendría la confirmación del inicio de sesión. Contempla la posibilidad de que pueda fallar, por ejemplo, por tratarse de un *nickname* que ya ha sido previamente registrado por otro usuario en el directorio. En caso de que el "login" se haya realizado con éxito, la respuesta por parte del directorio contendrá la *sessionKey* (número aleatorio asignado por el directorio a este cliente).

2. Observa el código de partida para modelar los mensajes en el proyecto *NanoFiles*.

1. Observa que `DirMessage` ya tiene un atributo `operation` de tipo `String`, que representa el campo del mensaje que indica el tipo de operación. Su valor por defecto es `DirMessageOps.OPERATION_INVALID` (operación no válida, usada para detectar que un mensaje está sin inicializar).
2. En la clase `DirMessageOps` verás que también hay una constante de clase `String` llamada `OPERATION_LOGIN` con el valor "login"; puedes utilizarla como el valor que debe tomar el atributo `operation` al crear el tipo de mensaje que debe ser enviado al directorio para iniciar sesión.

3. En `DirMessage` también hay otro atributo `nickname`, que podría usarse para albergar el valor de un campo utilizado para enviar un nombre de usuario en ciertas operaciones, tales como a la hora de iniciar sesión en el directorio.
3. Implementa la funcionalidad necesaria en los métodos `toString` y `fromString` de la clase `DirMessage`, de forma que ambos métodos sean capaz de manejar el tipo de mensaje que el cliente *NanoFiles* enviará al directorio para iniciar sesión.
4. Modifica tu implementación del método `loginToDirectory` de la clase `DirectoryConnector` para reemplazar el envío de una cadena de caracteres "en crudo" (sin formatear) por un mensaje de *login* bien formado, que siga el formato *field:value*, implementado en la clase `DirMessage`.
 1. Tras completar el boletín de prácticas anterior, tu código en este método actualmente debería enviar la cadena de caracteres "login&nickname" y comprobar que se recibe "loginok&NUM" como respuesta.
 2. En lugar de esto, a la hora de enviar un mensaje, debes crear un objeto `DirMessage` con su operación, establecer los campos necesarios (p.ej. establecer el valor del atributo *nickname*) usando el método **setter* adecuado, y finalmente convertirlo al String (con `.toString()`) cuyos bytes serán enviados en un datagrama.
5. Modifica tu implementación en el método `run` de la clase `NFDirectoryServer` para reemplazar el procesamiento del mensaje a partir de una cadena de caracteres "en crudo" (p.ej. "login&nickname") por el uso de métodos *getter* sobre un objeto `DirMessage` construido a mensajes de un mensaje bien formados en formato *field:value*.
 1. Tras completar el boletín de prácticas anterior, tu código del servidor actualmente debería comprobar que se ha recibido la cadena "login&nickname", en cuyo caso envía la cadena de caracteres "loginok&NUM".
 2. En lugar de esto, a la hora de recibir un mensaje, debes crear un objeto `DirMessage` a partir de la cadena de caracteres recibida en el datagrama (usando el método de clase `DirMessage.toString()`). Una vez creado dicho objeto, podrás acceder mediante métodos *getter* a los valores de los campos necesarios (p.ej. obtener el valor del atributo *nickname*).
6. Establece puntos de ruptura en los métodos `fromString` y `toString` que acabas de programar, y depura tu código ejecutando paso a paso tanto servidor (*Directory*) como cliente (**NanoFiles*), hasta comprobar que el servidor, a partir de los datos recibidos en el datagrama, construye un objeto `DirMessage` con valores idénticos al que creó el cliente antes de enviarlo como cadena de caracteres.
7. Sigue el `TODO` del método `buildResponseFromRequest` de la clase `NFDirectoryServer` para programa la lógica asociada al inicio de sesión en el directorio.
 1. Debe recibir como parámetro el mensaje que habrás construido previamente con `fromString`.
 2. Debe actuar de una u otra forma en función del tipo y datos mensaje recibido, consultando las estructuras de datos del servidor y actualizándolas en caso necesario. En el caso del inicio de sesión, deberá reflejar el registro del *nickname* con su *sessionKey* asignada al cliente, en caso de que el *login* haya sido exitoso (el *nickname* no esté duplicado).

3. Debe construir un mensaje `DirMessage`, con el tipo de operación y valores adecuados en función del resultado de la solicitud (éxito/fracaso). En el caso del mensaje de respuesta a una solicitud de inicio de sesión exitosa, el directorio deberá enviar la *sessionKey*.
 4. El mensaje retornado por este método a `run` deberá ser convertido a String y enviado como respuesta.
8. Termina de modificar tu implementación del método `logIntoDirectory` de la clase `DirectoryConnector` para reemplazar la recepción de una cadena de caracteres "en crudo" (sin formatear) por un mensaje de confirmación de inicio de sesión bien formado, que siga el formato *field:value*, implementado en la clase `DirMessage`.
1. Tras completar el boletín de prácticas anterior, tu código del cliente actualmente debería comprobar que se ha recibido la cadena "loginok&NUM", donde *NUM* es la *sessionKey*.
 2. En lugar de esto, a la hora de recibir el mensaje, debes crear un objeto `DirMessage` mediante `DirMessage.fromString()` y usarlo para comprobar su tipo y acceder, en su caso, al valor de la *sessionKey* codificada en el mensaje. `DirMessage.fromString()`
9. Una vez que hayas realizado los pasos anteriores, habrás completado exitosamente tu primera *conversación* con el directorio, con lo que ya deberías estar suficientemente familiarizado con la estructura básica del proyecto y los métodos involucrados en la comunicación UDP entre cliente y servidor. En este punto, puedes continuar diseñando e implementando el resto de mensajes necesarios para ir dotando de funcionalidad al programa:
1. Siguiendo con tu documento de texto plano, **diseña** el mensaje necesario para solicitar la lista de nombres de usuario registrados en el directorio, así como su respuesta correspondiente. Recuerda que una vez iniciada la sesión, todos los mensajes enviados al directorio deben contener la *sessionKey* que identifica al cliente.
 2. Contempla la posibilidad de que el directorio informe para cada *nickname* del listado si se trata o no de un par servidor (que está compartiendo sus ficheros en ese instante).
10. Teniendo en cuenta los pasos anteriores vistos para el comando `login`, adapta lo que sea necesario para programar la funcionalidad esperada del comando `logout`.
11. Para terminar de incorporar la funcionalidad mínima exigida al programa *NanoFiles*, implementa el envío y recepción de los mensajes adecuados así como la lógica requerida por el comando `userlist`.